

Production מערכת לניהול תקלות

Production Incident Management System

Full-Stack מסמך תכנון לפרויקט

React (JavaScript) -> Express / Node.js -> PostgreSQL
REST API | Testing | AWS deployment later

מטרת הפרויקט: ללמוד React ו-Express מהיסוד תוך בניית מערכת שנראית כמו כלי פנימי אמיתי של צוות Production Support.

גרסת תכנון ראשונית - אוגוסט 2026

1. מטרת הפרויקט והיקף העבודה

המערכת מדמה כלי פנימי לניהול תקלות בסביבת Production. היא תאפשר לפתוח Incidents, לשנות סטטוס וחומרה, לשייך בעלים, להוסיף עדכונים לאורך הטיפול, לסנן תקלות ולהציג תמונת מצב ניהולית.

תרחיש שימוש מרכזי

- משתמש פותח Incident חדש ומזין כותרת, תיאור, חומרה, שירות מושפע ובעלים.
- התקלה עוברת מ Resolved Open, Investigation.
- בכל שלב ניתן להוסיף עדכון ל Timeline של התקלה.
- במסך הראשי ניתן לחפש, לסנן ולמיין תקלות.
- Dashboard מציג מספר תקלות פתוחות, קריטיות, תקלות שנסגרו וזמן טיפול ממוצע.

טכנולוגיות

תפקיד	טכנולוגיה	שכבה
מסכים, טפסים, State, ניווט וקריאות API	React + JavaScript + Vite	Frontend
REST API, Validation, לוגיקה עסקית וטיפול בשגיאות	Node.js + Express	Backend
שמירת Incidents, Updates, ובהמשך משתמשים והרשאות	PostgreSQL	Database
בדיקות ידניות ואוטומטיות	Postman ובהמשך Playwright / API tests	Testing
EC2, IAM, CloudWatch S3 ובמידת הצורך	AWS בשלב מאוחר	Deployment

MVP - הגרסה הראשונה שחייבת לעבוד

- רשימת Incidents עם נתונים אמיתיים מהשרת.
- יצירת Incident חדש.
- צפייה בפרטי Incident יחיד.
- שינוי Status ו-Severity.
- סינון לפי Status ו-Severity.
- שמירה ב-PostgreSQL וטיפול בסיסי בשגיאות.

עיקרון חשוב

לא בונים מיד Authentication, גרפים, Docker או AWS. קודם משלימים זרימה אחת מלאה: React -> Express -> PostgreSQL -> JSON -> React.

2. מבנה הפרויקט

```
incident-management-system/
|-- client/
|-- server/
|-- docs/
|-- .gitignore
`-- README.md
```

תיקיה / קובץ	אחריות
client/	אפליקציית React וכל קוד ה-Frontend.
server/	Express, לוגיקה עסקית, גישה למסד הנתונים ו-API.
/docs	תרשימי ארכיטקטורה, צילומי מסך ותיעוד API.
README.md	הסבר על הפרויקט, הוראות הרצה, ארכיטקטורה ויכולות.
gitignore.	מונע העלאת node_modules, קובצי .env וקבצים זמניים.

מבנה ה-Frontend הסופי

```
client/
|-- public/
|-- src/
|   |-- api/
|   |   |-- incidentsApi.js
|   |-- components/
|   |   |-- layout/
|   |   |   |-- Header.jsx
|   |   |   |-- Sidebar.jsx
|   |   |-- incidents/
|   |   |   |-- IncidentCard.jsx
|   |   |   |-- IncidentList.jsx
|   |   |   |-- IncidentForm.jsx
|   |   |   |-- IncidentFilters.jsx
|   |   |   |-- IncidentStatusBadge.jsx
|   |   |   |-- IncidentUpdateForm.jsx
|   |   |-- common/
|   |   |   |-- LoadingMessage.jsx
|   |   |   |-- ErrorMessage.jsx
|   |   |   |-- EmptyState.jsx
|   |-- pages/
|   |   |-- DashboardPage.jsx
|   |   |-- IncidentsPage.jsx
|   |   |-- IncidentDetailsPage.jsx
|   |   |-- CreateIncidentPage.jsx
|   |   |-- NotFoundPage.jsx
|   |-- hooks/
|   |   |-- useIncidents.js
|   |-- utils/
|   |   |-- formatDate.js
|   |   |-- calculateIncidentAge.js
|   |-- App.jsx
|   |-- main.jsx
|   |-- index.css
|-- .env
|-- package.json
`-- vite.config.js
```

3. אחריות הקבצים בצד React

קובץ / אזור	מה הוא עושה
/pages	כל Page הוא מסך שלם שמחבר כמה Components ומנהל את זרימת הנתונים של אותו מסך.
DashboardPage.jsx	מציג סיכום: תקלות פתוחות, קריטיות, נסגרו היום וזמן טיפול ממוצע.
IncidentsPage.jsx	רשימה, חיפוש, סינון ומיון.
IncidentDetailsPage.jsx	פרטי Incident, Timeline, שינוי Status והוספת Update.
CreateIncidentPage.jsx	מסך יצירת Incident חדש.
/components/incidents	רכיבים לשימוש חוזר הקשורים לתקלות.
IncidentCard.jsx	כרטיס קצר של תקלה אחת. מקבל incident דרך Props.
IncidentList.jsx	מקבל Array ומציג IncidentCard עבור כל איבר באמצעות map.
IncidentForm.jsx	טופס ליצירה או עריכה של Incident.
IncidentFilters.jsx	חיפוש וסינון לפי Status, Severity, Service וסדר מיון.
api/incidentsApi.js	כל קריאות fetch לשרת במקום לפזר כתובות API בתוך Components.
hooks/useIncidents.js	Custom Hook אופציונלי שירכז incidents, loading, error ו-reload-l.
/utils	פונקציות JavaScript טהורות כמו פורמט תאריך וחישוב גיל התקלה.

זרימת מידע בצד הלקוח

```
User action
-> React event handler
-> incidentsApi.js (fetch)
-> Express REST API
-> JSON response
-> setState
-> React re-render
```

מצבי UI שחייבים להופיע

- Loading - בזמן שמחכים לתשובת השרת.
- Error - כאשר קריאת API נכשלת.
- Empty - כאשר אין Incidents להצגה.
- Success - כאשר הפעולה הסתיימה והמידע הוצג או נשמר.

לא ליצור מוקדם מדי

את useIncidents.js יוצרים רק כשה-fetch וה-State מתחילים לחזור בכמה Pages. בהתחלה עדיף לראות את כל הזרימה בתוך IncidentsPage.jsx ולהבין אותה.

4. מבנה ה-Backend ב-Express

```
server/
|-- src/
|   |-- config/
|   |   |-- database.js
|   |-- controllers/
|   |   |-- incidentsController.js
|   |   |-- incidentUpdatesController.js
|   |   |-- dashboardController.js
|   |-- routes/
|   |   |-- incidentsRoutes.js
|   |   |-- incidentUpdatesRoutes.js
|   |   |-- dashboardRoutes.js
|   |-- services/
|   |   |-- incidentsService.js
|   |   |-- dashboardService.js
|   |-- repositories/
|   |   |-- incidentsRepository.js
|   |   |-- incidentUpdatesRepository.js
|   |-- middleware/
|   |   |-- errorHandler.js
|   |   |-- notFound.js
|   |   |-- validateIncident.js
|   |-- utils/
|   |   |-- logger.js
|   |-- app.js
|   |-- server.js
|-- database/
|   |-- schema.sql
|   |-- seed.sql
|-- tests/
|   |-- incidents.test.js
|-- .env
|-- .env.example
-- package.json
```

הזרימה של בקשת HTTP

Route -> Controller -> Service -> Repository -> PostgreSQL
<- JSON Response <-

שכבה	אחריות
Route	מגדיר Method ו-URL ומעביר את הבקשה ל-Controller המתאים. לא מכיל SQL.
Controller	קורא req.params, req.query ו-req.body; מחזיר Status Code ו-JSON.
Service	מכיל חוקים עסקיים, למשל מה קורה כאשר Incident עובר ל-Resolved.
Repository	מבצע Queries מול PostgreSQL ומחזיר נתונים לשכבת Service.
Middleware	Validation, טיפול מרכזי בשגיאות ו-404.
app.js	מגדיר Express, middleware ו-routes.
server.js	מפעיל את האפליקציה ומאזין ל-Port.
database.js	יוצר Connection Pool ל-PostgreSQL באמצעות DATABASE_URL.

למה ההפרדה טובה

כך כל קובץ עונה על שאלה אחת: איזה URL קיים, איך מטפלים ב-HTTP, מה החוק העסקי ואיך ניגשים למסד. זה מקל על Debugging ובדיקות.

5. תכנון מסד הנתונים

בגרסה הראשונה מספיקות שתי טבלאות. מערכת משתמשים והרשאות תתווסף רק לאחר שה-MVP עובד.

טבלת incidents

שדה	סוג	משמעות
id	SERIAL PRIMARY KEY	מזהה ייחודי.
title	VARCHAR(150)	כותרת קצרה לתקלה.
description	TEXT	תיאור מלא של התקלה.
severity	VARCHAR(20)	.Low / Medium / High / Critical
status	VARCHAR(30)	.Open / Investigating / Resolved
affected_service	VARCHAR(100)	השירות או המערכת המושפעים.
assigned_to	VARCHAR(100)	בעלים זמני; בהמשך יהפוך ל-user_id.
created_at	TIMESTAMP	מועד פתיחת התקלה.
updated_at	TIMESTAMP	מועד העדכון האחרון.
resolved_at	TIMESTAMP	מועד סגירה, או NULL כשהתקלה פתוחה.

טבלת incident_updates

שדה	סוג	משמעות
id	SERIAL PRIMARY KEY	מזהה העדכון.
incident_id	INTEGER + FOREIGN KEY	מקשר את העדכון ל-Incident.
message	TEXT	תוכן העדכון.
created_by	VARCHAR(100)	מי כתב את העדכון.
created_at	TIMESTAMP	מועד יצירת העדכון.

קשר בין הטבלאות

incidents (1) ----- (many) incident_updates

One incident can have many timeline updates.

Deleting an incident can delete its updates with ON DELETE CASCADE.

חוקים עסקיים בסיסיים

- כאשר status משתנה ל-Resolved, השרת ממלא resolved_at.
- כאשר Incident חוזר ל-Open, אפשר לאפס resolved_at.
- Critical Incident חייב לכלול affected_service.
- Status ו-Severity חייבים להיות מתוך רשימת ערכים מותרת.
- כל שינוי Status יכול ליצור Update אוטומטי ב-Timeline.

6. תכנון ה-REST API

Method ונתיב	מטרה
GET /api/incidents	קבלת כל התקלות. יתמוך בהמשך ב-query parameters לסינון ומיון.
GET /api/incidents/:id	קבלת Incident יחיד לפי ID.
POST /api/incidents	יצירת Incident חדש.
PATCH /api/incidents/:id	עדכון פרטים כמו title, description, severity או assigned_to.
PATCH /api/incidents/:id/status	שינוי Status בלבד והפעלת החוקים העסקיים המתאימים.
DELETE /api/incidents/:id	מחיקה לצורך הלמידה; במערכת אמיתית ייתכן שנעדיף Archive.
GET /api/incidents/:id/updates	קבלת Timeline של Incident.
POST /api/incidents/:id/updates	הוספת עדכון חדש ל-Timeline.
GET /api/dashboard/summary	קבלת נתוני סיכום למסך Dashboard.

Query Parameters לדוגמה

```
GET /api/incidents?status=open&severity=critical
GET /api/incidents?search=database&sort=created_at&order=desc
```

Body לדוגמה - יצירת Incident

```
{
  "title": "Payment API unavailable",
  "description": "Requests return HTTP 500",
  "severity": "critical",
  "affectedService": "payments-api",
  "assignedTo": "Shon"
}
```

Status Codes עיקריים

קוד	מתי משתמשים
200	בקשת GET או PATCH הצליחה.
201	משאב חדש נוצר בהצלחה.
400	Body או ערך לא תקין.
404	Incident לא נמצא או URL לא קיים.
409	פעולה מתנגשת עם מצב קיים, לפי הצורך.
500	שגיאה לא צפויה בשרת.

כלל ל-Controller

ה-Controller מטפל ב-HTTP בלבד. הוא לא כותב SQL ולא אמור להכיל את כל חוקי המערכת.

7. סדר הבנייה המומלץ

מה לומדים	מה בונים	שלב
Components, Props, State, Events, map, Conditional Rendering ו-Forms	שלושה Incidents בתוך Array, כרטיסים, רשימה, סינון וטופס.	1. React עם מידע זמני
Routes, Controllers, req/res ו-Status Codes.	GET, GET by ID ו-POST; הנתונים נשמרים זמנית ב-Array בשרת.	2. Express בסיסי
Client- וזרימת async/await, useEffect ו-Server	fetch, loading, error יצירה ורענון רשימה.	3. חיבור React ל-Express
SQL, הפרדת שכבות וטיפול בשגיאות DB.	Queries ו- schema.sql, Pool, Repository פרמטריים.	4. PostgreSQL
לוגיקה עסקית, Query Parameters ו-State מורכב יותר.	Status, Timeline, Search, Filters ו-Dashboard.	5. יכולות אמיתיות
Production, בדיקות, Security ו-Documentation.	Authentication, Roles, Tests, Deployment, CloudWatch ו-AWS	6. השלמות מקצועיות

מבנה מינימלי להתחלה

```
incident-management-system/
|-- client/
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- App.jsx
|   |   |-- main.jsx
|   |-- server/
|   |   |-- src/
|   |   |   |-- routes/
|   |   |   |   |-- incidentsRoutes.js
|   |   |   |-- controllers/
|   |   |   |   |-- incidentsController.js
|   |   |   |-- app.js
|   |   |   |-- server.js
|   |-- package.json
```

המשימה הראשונה

- ליצור פרויקט React ב-JavaScript עם Vite.
- להגדיר Array של שלושה Incidents מזויפים.
- לבנות IncidentCard ו-IncidentList.
- להציג Status, Severity, Service, Owner וזמן פתיחה.
- להוסיף סינון בסיסי לפי Status.
- רק לאחר שזה ברור ועובד - להתחיל את Express.

Definition of Done לשלב הראשון

המסך מציג שלוש תקלות מתוך Array, כל תקלה מוצגת דרך Component לשימוש חוזר, והסינון משנה את הרשימה בלי שינוי ידני של ה-DOM.

8. הרחבות לאחר ה-MVP

יכולות מוצר

- Authentication והרשאות: Admin, Responder ו-Viewer.
- Audit Log של שינויים חשובים.
- SLA וחישוב זמן עד תגובה וזמן עד פתרון.
- Service catalog ורשימת צוותים.
- גרפים של Incidents לפי שירות, חומרה ותקופה.
- notifications-I Comments, attachments

בדיקות ואיכות

- Postman Collection לכל ה-endpoints.
- בדיקות API אוטומטיות למסלולים העיקריים.
- Validation בצד השרת ובצד הלקוח.
- בדיקות React עבור מצבי Error, Loading ו-Empty.
- טיפול מרכזי בשגיאות ולוגים קריאים.

AWS בשלב הסופי

```
Internet
-> EC2 / application host
-> Express API
-> PostgreSQL

CloudWatch:
- EC2 CPU metrics
- Alarm + SNS email
- application logs later
```

מה להציג ב-GitHub

- README ברור עם מטרת המערכת והבעיה שהיא פותרת.
- תרשים ארכיטקטורה ותרשים מסד נתונים.
- צילומי מסך של Dashboard, רשימה, Details וטופס.
- רשימת Endpoints ודוגמאות Request/Response.
- הוראות התקנה והרצה מקומית.
- סעיף Troubleshooting: בעיות שנתקלת בהן ואיך פתרת אותן.

ניסוח אפשרי לקורות החיים לאחר שהפרויקט עובד

```
Production Incident Management System
- Built a full-stack incident management application with React, Express and PostgreSQL.
- Implemented REST APIs, filtering, status workflows and incident timelines.
- Added validation, centralized error handling and production-style troubleshooting flows.
- Documented the architecture, API and deployment process.
```

הקו המנחה

הפרויקט צריך להיות קודם כל עובד וברור. ארכיטקטורה יפה ו-README טוב חשובים, אבל הם לא מחליפים זרימה מלאה שאפשר להדגים בלחיצה.